Python Assignment
Topic-6: Generators and Iterators

Q1. Understanding how generators work in Python.

Ans.

Generators are special functions in Python that are used to generate values one by one instead of returning all values at once. They use the 'yield' keyword instead of 'return'. When a generator function is called, it does not execute completely but returns an iterator. Each time we use it, it gives the next value. Generators help in saving memory because they do not store all values at once. They are useful when working with large data. Generators make programs efficient and faster.

Example:

```
def gen():
    yield 1
    yield 2
    yield 3

for i in gen():
    print(i)
```

Q2. Difference between yield and return.

Ans.

The 'return' statement is used to return a value and end the function completely. Once return is used, the function stops execution. The 'yield' statement is used in generators to return a value but it does not stop the function permanently. It pauses the function and resumes from the same point next time. Return gives only one value, while yield can give multiple values one by one. Yield is mainly used for memory-efficient programming.

Example:

```
def test():
    yield 10
    yield 20
```

```
for i in test():
    print(i)
```

Q3. Understanding iterators and creating custom iterators.

Ans.

Iterator is an object that allows us to access elements one by one. In Python, lists, tuples, and strings are iterable objects. An iterator is created using iter() function and elements are accessed using next() function. Custom iterators can be created using classes with special methods like __iter__() and __next__(). Iterators are useful when working with loops and large data. They help in processing data step by step. This makes programs efficient and easy to manage.

Example:

```
lst = [1,2,3]
it = iter(lst)
print(next(it))
print(next(it))
```